

DocPilot AI

Complete Antigravity Implementation Plan

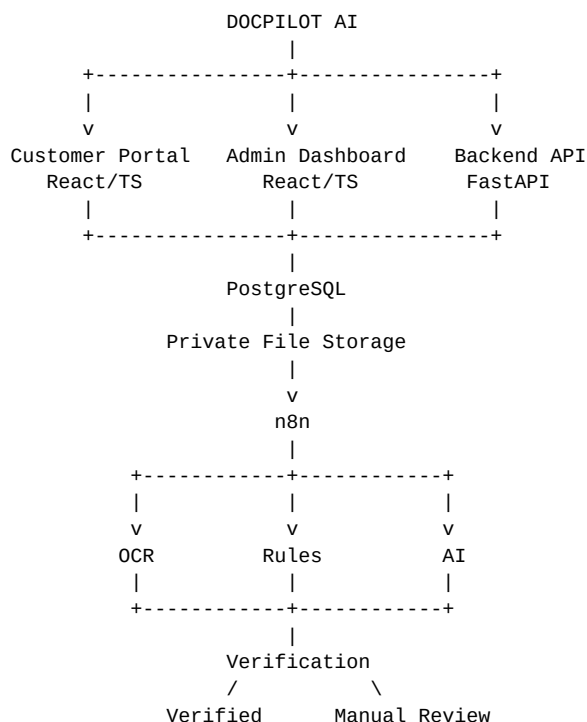
Frontend + Backend + Database + OCR + Rules Engine + AI + n8n + Admin Dashboard

This plan converts the supplied DocPilot AI workflow into a structured development roadmap for implementation using Antigravity. The source workflow establishes the core sequence: consent, secure upload, encrypted temporary storage, OCR, privacy masking, rules-first verification, AI escalation when required, manual review, reminders, completion, retention and deletion.

Source basis

The uploaded workflow material identifies the customer portal, admin dashboard, OCR/verification pipeline, secure token model, supported documents, state machines, API concepts and retention/deletion flow. This implementation plan keeps those concepts and organizes them into build phases.

1. Target Architecture



2. Antigravity Development Strategy

Do not ask Antigravity to build the complete application in one prompt. Work phase-by-phase. At every phase instruct it to inspect the repository, explain its findings, implement the requested scope, run tests/build/type checks, list changed files and report remaining issues.

- Phase 1 — Foundation
- Phase 2 — PostgreSQL database and models
- Phase 3 — FastAPI backend and API
- Phase 4 — Customer secure upload portal

- Phase 5 — Admin dashboard
- Phase 6 — OCR service
- Phase 7 — Privacy Gateway and deterministic Rules Engine
- Phase 8 — AI escalation
- Phase 9 — n8n integration
- Phase 10 — Security, retention, testing and deployment

3. Recommended Repository Structure

```
docpilot-ai/  
  frontend/  
    customer-portal/  
    admin-dashboard/  
  backend/  
    app/  
      api/  
      models/  
      schemas/  
      services/  
      security/  
      storage/  
      verification/  
    tests/  
  ocr-service/  
  n8n/  
  database/  
    migrations/  
    schema/  
  docs/
```

4. Phase 1 — Foundation Prompt for Antigravity

You are the lead software architect and full-stack engineer for a production application called DocPilot AI.

Build incrementally. Do not implement the entire system in one step.

Architecture:

- React + TypeScript for frontend
- Vite
- Tailwind CSS
- Python FastAPI backend
- PostgreSQL
- SQLAlchemy
- Pydantic
- Alembic
- n8n as workflow orchestration
- private object storage abstraction

Create:

```
/frontend/customer-portal  
/frontend/admin-dashboard  
/backend  
/ocr-service  
/n8n  
/database  
/docs
```

First task only:

1. Create repository structure.
2. Create FastAPI skeleton.
3. Create customer portal skeleton.
4. Create admin dashboard skeleton.
5. Configure PostgreSQL.
6. Create Docker Compose.
7. Create environment templates.
8. Create README explaining service communication.

Do not implement OCR, AI, n8n workflows or authentication yet.

Inspect first, implement second, then run checks and report results.

5. Phase 2 — Database

Use PostgreSQL, SQLAlchemy and Alembic.

Entities:

Customer

Document

UploadSession

ConsentRecord
VerificationResult
ManualReview
Reminder
PrivacyRequest
AuditRecord

Customer:
id, customer_id, name, email, status, required_documents,
consented_at, completed_at, delete_after, created_at, updated_at

Document:
id, customer_id, document_type, file_name, storage_key,
file_size, mime_type, ocr_status, verification_status,
verification_method, confidence, review_reason,
uploaded_at, verified_at, scheduled_deletion_at, deleted_at

Customer states:
new -> consented -> active -> completed -> deleted

OCR states:
waiting -> processing -> completed | failed

Verification:
pending | verified | rejected | manual_review

Invariant:
pending_documents = required_documents - verified_documents

Create migrations, indexes, constraints, relationships and database tests.

6. Phase 3 — Backend API

Base: /api/v1

GET /customers
GET /customers/{id}
GET /customers/{id}/documents
POST /documents/{id}/review
GET /documents/{id}/file
POST /portal/sessions
GET /portal/u/{token}
POST /portal/u/{token}/upload
POST /webhooks/n8n
POST /privacy/requests

Requirements:

- random expiring customer session tokens
- customer sees only own pending documents
- validate MIME type, extension, size and document type
- private storage only
- no public bucket URLs
- short-lived secure document access
- admin authentication/authorization
- Pydantic validation
- structured errors
- API tests
- isolate n8n behind a service layer

7. Phase 4 — Customer Portal

The portal should be simple and trustworthy. Customers upload documents gradually and should see only their pending requirements. The source workflow explicitly supports partial uploads and recalculates pending documents after each verified upload.

Pages:

1. Secure Session
2. Customer Document Dashboard
3. Document Upload
4. Upload Progress
5. Verification Status
6. Consent / Privacy
7. Completion

Document states shown to customer:

Verified
Upload
Processing
Failed / Retry

Do not expose:

- database IDs
- storage keys
- internal rules
- AI prompts
- reviewer notes
- raw OCR data

Upload:

drag/drop, browse, validation, progress, success, processing, failure, retry

8. Phase 5 — Admin Dashboard

Sidebar:

Overview
Customers
Documents
Manual Reviews
Reminders
Audit Log
Settings

Overview metrics:

- total customers
- total documents
- verified documents
- pending documents
- processing documents
- manual reviews
- OCR failures
- completion rate

Documents page:

- search
- customer filter
- document type filter
- OCR status filter
- verification status filter
- date filter

Table:

Customer	Document	Uploaded	OCR	Verification	Confidence	Review	Deletion Date	Actions
----------	----------	----------	-----	--------------	------------	--------	---------------	---------

Customer detail:

- customer information
- case status
- consent
- required/received/pending documents
- timeline

- retention/deletion date

Manual review:

- secure document preview
- masked evidence
- verification result
- confidence
- review reason
- method
- Approve / Reject / Request Resubmission

Mask sensitive information and never expose storage keys.

9. Dashboard Visual Direction

The finished dashboard should look like a modern enterprise document-management control center, not a marketing website. Use a dark/neutral professional shell or a restrained light enterprise interface, strong typography, clean tables, clear spacing, subtle animations and consistent status indicators.

State	Visual treatment
Verified	Green status badge
Manual Review	Amber status badge
Rejected / Failed	Red status badge
Pending	Neutral gray badge
Processing	Animated/progress indicator
Deleted	Muted/struck-through state

10. Phase 6 — OCR Service

Pipeline:

Upload

- > validation
- > preprocessing
- > OCR
- > document classification
- > field extraction
- > normalized OCR result
- > Privacy Gateway

Provider abstraction:

OCRProvider

- > PaddleOCRProvider
- > MockOCRProvider

Normalized output:

document_type
holder_name
issuer
dates
expiry_date
document_number_masked
structural_markers
extracted_fields
ocr_confidence
ocr_errors

Handle:

- unreadable documents
- OCR failure
- unsupported documents
- low confidence
- rotated images
- multi-page PDFs

11. Supported Document Types

- PAN Card
- Aadhaar Card
- Bank Statement
- Salary Slip / Payslip
- Passport
- Voter ID
- Driving Licence
- Cancelled Cheque
- ITR / ITR Acknowledgement
- Udyam Registration Certificate
- Shop & Establishment Certificate
- FSSAI Certificate
- Electricity / Utility Bill

12. Phase 7 — Privacy Gateway + Rules Engine

OCR

↓

Privacy Gateway

↓

Rules Engine
↓
Is evidence sufficient?
■■■ YES -> decision
■■■ NO -> redacted evidence -> AI

Privacy masking:
- Aadhaar
- PAN
- bank account numbers
- phone numbers
- DOB
- address

Rules:
1. document type
2. structural markers
3. required fields
4. expiry
5. name compatibility
6. risk flags
7. OCR quality
8. document validity

Return:
result
confidence
reasons[]
evidence[]
requires_ai
requires_manual_review

Critical rule:
Use deterministic rules first.
Do not call an LLM when rules are sufficient.

13. Phase 8 — AI Escalation

AI is a controlled fallback, not the primary verifier. When deterministic rules are uncertain, the system should send only redacted evidence to the configured AI provider. The AI result is then returned to the verification layer and may still lead to manual review.

Rules uncertain
↓
PII masking
↓
Redacted evidence
↓
AI provider
↓
Verification result
↓
Verified / Pending / Manual Review

14. Phase 9 — n8n Integration

```
Backend
|
| upload / verification / reminder / retention events
v
n8n
|
+-- consent email
+-- OCR orchestration
+-- verification workflow
+-- AI escalation
+-- manual review notification
+-- 3/7/14-day reminders
+-- completion
+-- retention/deletion
|
v
Backend webhooks
```

Important:

Inspect the existing n8n JSON before modifying it.

Reuse existing workflow logic where appropriate.

Do not duplicate business logic unnecessarily in multiple systems.

15. Phase 10 — Security

- Authentication and authorization
- Random expiring upload tokens
- Token revocation/expiration
- Private file storage
- Short-lived signed document access
- Upload MIME/extension/size validation
- Malicious file/safety checks
- PII masking
- SQL injection protection
- XSS protection
- CSRF protection where applicable
- Rate limiting
- Webhook authentication
- Secret management
- Audit logging
- Consent withdrawal
- Customer deletion requests
- Retention enforcement
- Permanent deletion

16. End-to-End Testing

Test scenario 1:

Customer created

-> consent

-> secure link

-> upload PAN
-> OCR
-> rules
-> verified
-> pending recalculated

Test scenario 2:
Upload multiple documents
-> one verified
-> one manual review
-> one failed
-> pending list remains correct

Test scenario 3:
Expired token
-> upload denied
-> new session generated

Test scenario 4:
Rules uncertain
-> PII masked
-> redacted evidence sent to AI
-> result stored

Test scenario 5:
All documents verified
-> customer completed
-> 7-day retention starts
-> raw files deleted
-> minimal audit retained

Test scenario 6:
Consent withdrawn
-> processing/reminders stop

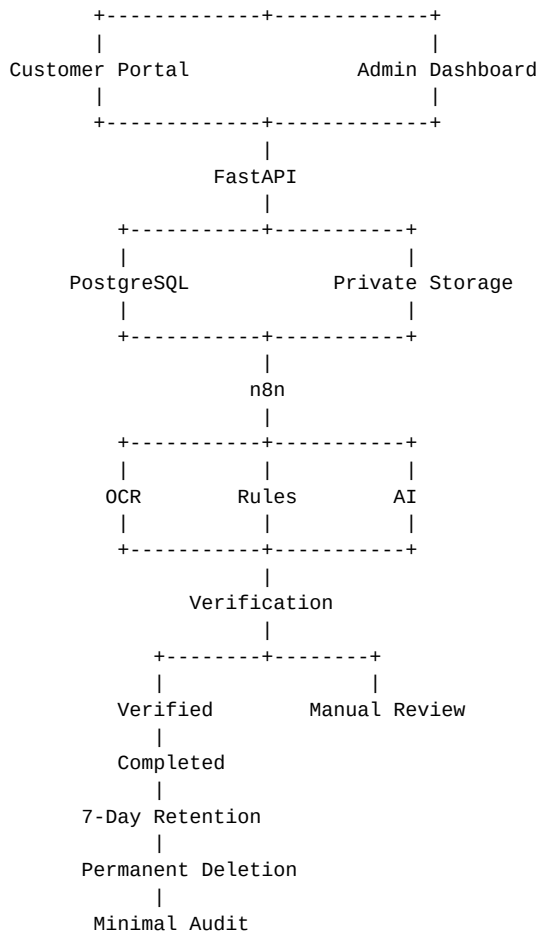
Test scenario 7:
Customer requests deletion
-> identity/confirmation
-> deletion
-> audit update
-> confirmation

17. Antigravity Working Rules

- Never implement the entire application in one prompt.
- Inspect existing code before editing.
- Do not invent APIs that conflict with the database.
- Keep frontend, backend, OCR and workflow responsibilities separated.
- Do not replace the existing n8n workflow without inspecting it.
- Run tests after each major phase.
- Run TypeScript/build checks for frontend changes.
- Run Python tests and lint/type checks for backend changes.
- Do not leave fake mock data connected after the real API exists.
- Never expose raw PII or private storage credentials.
- Document architectural decisions and environment variables.
- Keep secrets in environment/secret management, never source code.

18. Final System

DOCPILOT AI
|



19. Implementation Order

1. Foundation and repository
2. Database and state machines
3. FastAPI backend
4. Customer secure portal
5. Admin dashboard
6. OCR service
7. Privacy Gateway
8. Rules Engine
9. AI escalation
10. n8n integration
11. Security hardening
12. End-to-end testing
13. Deployment and monitoring

20. Source Workflow Alignment

The supplied workflow describes secure upload sessions, partial uploads, supported document OCR, PII masking, rules-first verification, manual review, reminders, completion, seven-day retention, deletion and minimal audit. The implementation plan above maps those concepts into separate application components while retaining n8n as the orchestration layer.